The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
PRADEEP N	24MIP10033
Slot	**Date of Submission**
E11	30-03-2026

1. Introduction

Open source is the collaborative foundation of modern technology. Instead of one company restricting software with a price tag, these digital tools, including smartphone engines and supercomputer systems, are developed by a worldwide community that shares, improves, and provides the code for everyone’s benefit..

You will choose one real open-source software project, something you can download, read about, and interact with on Linux, and conduct a structured audit of it. This audit should go beyond just answering 'what does it do?' You need to explore why it exists, who built it, what freedoms its license gives you, what philosophical values it represents, and how it has influenced the world around it.

Along with the written report, you will create five shell scripts that showcase practical Linux skills. These scripts do not need to be impressive; instead, they should demonstrate your understanding of how open-source tools function at the command line and how automation relates to the idea of sharing and transparency.

It is used in a wide range of fields, including web development, data analysis, artificial intelligence, automation, and scientific computing. What makes Python especially interesting in the context of open source is not just what it does, but how it has evolved—through contributions from developers all over the world.

Python was invented by Guido van Rossum and first released in 1991. Since then, it has grown far beyond a single developer’s project into a global collaboration. Its development is now guided by the Python Software Foundation, which ensures that the language remains free, open, and accessible to everyone. Anyone can download Python, study how it works, modify it, and even contribute to its future versions.

This report goes beyond simply describing Python’s features. It looks at why Python exists, how it is developed, and what freedoms its open-source license provides. By examining Python in this way, the goal is to better understand not just the software itself, but the broader philosophy of open source—where transparency, collaboration, and shared knowledge drive innovation.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: _____PYTHON_____

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

The "Broken" Reality Before Python :

C was very "hard": To build effective tools, you would need to program in C, but C was very difficult to use because it required the application programmer to do a lot of very low-level memory management in order for their application to be successful. For example, even the task of reading two lines from a file could cause a system crash if the programmer didn't have the foresight to allocate enough memory to perform the read operation. It was like building your own engine just to be able to drive to the grocery store.

ABC was too "Closed": Guido van Rossum had worked on a language called **ABC**, which was designed to be beautiful and easy for non-scientists. It was elegant, but it was a "black box." It couldn't talk to the rest of the computer easily, it didn't have a way to save files simply, and it was hard to distribute.

A2. The license — what it actually says

- In the world of free software, "free" isn't about the price; it’s about having total control over your tools. To be truly free, a license must give you four specific rights. First, you must be allowed to run the program for any reason you want.Can a company take this software, modify it.

- Whether a company can take Python, tweak it, and sell it secretly depends on the license. Since Python uses a permissive license, a company is actually allowed to take the Python source code, add their own private features or optimizations, and sell that version as a closed product

- The main difference is how much you want to "force" others to share. The **GPL** is like a "pay it forward" rule: if you use it, you must keep your code open for everyone else. The **MIT** license is much simpler and basically says, "Do whatever you want with this, just give me credit." If I were building a new data tool or a script today, I’d choose the **MIT** license.

A3. The ethics of open source

The biggest reason to make everything open source is **honesty**. When software is "closed," you just have to trust that the company isn't doing anything shady with your data or leaving big security holes open. If the code is public, anyone can check it, find bugs, and fix them.

- The real ethical problem starts when a company becomes a "parasite." This happens when a business takes a successful open-source project, puts a fancy wrapper around it, and sells it, but never shares their fixes or contributes money or code back to the original creators.

- In the software world, this phrase means that nobody has to start from zero anymore. Think about building an app: you don’t have to write the code that tells a computer how to connect to the internet or how to display a button on a screen.

- Open source is like a massive library where the books are free to edit. It speeds everything up because it stops people from "reinventing the wheel." Instead of ten different companies building ten different versions of the same basic tool,

- they all contribute to one great tool. This lets everyone spend their energy on the creative, "new" parts of their project rather than the boring basics.

- Even with those risks, most of the modern world runs on open source. It shows that sharing actually creates more progress than keeping secrets. By making the "giants" taller every year, open source gives every new developer a better starting point to build the next big thing.

Part B — Linux Footprint (Unit 2)

- Since I'm using **Ubuntu**, I've noticed that Python doesn't just sit in one single folder like it might on Windows. Instead, it's spread out across the system in a way that keeps things organized but can be a bit confusing if you don't know where to look.

- Finally, there is the "expert" method: **compiling from source**. This is where you download the raw "recipe" of Python (the actual code it's written in) and tell your computer to build the software from scratch.

- When I type python3 in my terminal, the system looks in **/usr/bin/python3**. This isn't usually the actual file, but a "shortcut " , that points to the specific version I have installed, like python3.12.\

- The core parts of the language are stored in **/usr/lib/python3**. If I install extra libraries for a project—like something for data engineering—they usually go into **/usr/local/lib/python3.x/dist-packages/.**

- One thing that surprised me is that Python doesn't have a main log file in /var/log like other system services do .

- if I'm running a Python script as a background task using something like systemd, then those logs get sent to the Ubuntu system log.

- Since I'm using Python as my software, I've learned that it doesn't automatically run as a "service" the way a database like MySQL does. Python is just a language, so it only runs when I tell it to execute a script.

  - Update When I'm using **Ubuntu**, the way I get security patches for **Python** is actually a very smooth, automated process. It's not like some software where I have to manually go to a website and download an installer every time there's a bug.

Part C — The FOSS Ecosystem (Units 3 & 4)

.

- When I look at my **Ubuntu** system, I've realized that Python isn't just a single file that works on its own. It's actually built on a foundation of other famous open-source tools ,

- Even though I write in Python, the computer actually understands **C**. Python itself is mostly written in a language called C.

- It’s hard to imagine the tech world without Python today. Because its code is open for anyone to use and change, it hasn't just been a tool—it has been the "spark" for entirely new industries.

- the language is so easy to read, scientists and mathematicians—who aren't always professional programmers—started building massive tools on top of it. Engines like **TensorFlow** .

- **PyTorch** (which run most AI today) and libraries like **Pandas** (which I use for data engineering) turned Python into a powerhouse that can handle millions of rows of data in seconds.

- In the traditional stack, PHP was the go-to language for making websites interactive (like handling a login or a shopping cart). Today, many developers (including me) swap out PHP for **Python**.

- Python has made web development much more "human." Instead of wrestling with complex, messy code, I can use Python to build powerful web tools that are easy to read and maintain.

- this setup is the secret to Python's success. Because no single company owns it, the language stays focused on what’s best for the community, not just what makes the most mone.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source	Proprietary Alternative
Cost	Totally free. I have it on my Ubuntu setup right now. No fees, even if I use it for work later.	Very expensive. A license can cost thousands.
Security (who can audit the code?)	Open for anyone to see. If there's a bug, the community finds and fixes it fast.	A total mystery. You have to trust that their internal team caught everything.
Support and reliability	**Community-Powered.** If I'm stuck, I hit Stack Overflow or Discord	**Professional & Guaranteed.** Since you pay, you get a dedicated support line
Freedom to modify	**Total Control**	Locked Down.

Community vs corporate control	**Community-Led.** Changes are driven by what developers actually need.	**Corporate-Led.** Decisions are made by MathWorks.
Your overall verdict	**The "Universal Tool."** Best for Data Engineering, AI, and web projects. It’s free, flexible,	**The "Specialized Powerhouse."** Best for high-end hardware simulation

If I had to pick one for a real-world project, I’d choose **Python** every time. It’s not just about the $0 price tag; it’s about the fact that it connects to everything. Whether I'm pulling data from a SQL database or pushing it to a cloud server, there's already a tool in Python built to do it

As for contributing, I’m definitely in. One of the best things about this assignment was realizing that Python stays great because regular people actually bother to fix the things that are broken

Since I’m already using it for my cryptography and stats labs, I’d love to start small—maybe by helping with documentation or fixing a minor bug in a data library. There’s something really cool about knowing that instead of just complaining about a piece of software, I can actually jump in and help make it better for the next person..

4. Shell Script Tasks

.

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Starter structure

```
prade@PAVIPRADEEP:~$ cat scripts.sh
    #!/bin/bash
    # Script 1: System Identity Report
    # Author: PRADEEP N | Software Focus: Python
    # Using: variables, echo, command substitution, and box-formatting

    # Setting up project-specific identifiers
    NAME="Pradeep N"                # Your name for the report header
    SOFTWARE_CHOICE="Python"        # The open-source tool we are auditing


    # Gathering live environmental data from the system
    # These lines capture the technical "DNA" of the current machine
    KERNEL=$(uname -r)                    # Grabbing the specific kernel build version
    USER_NAME=$(whoami)                   # Identifying the person currently at the terminal
    HOME_DIR=$HOME                        # Locating the path to the user's primary folder
    UPTIME=$(uptime -p)                   # Checking how long the system has been running
    CURRENT_DATE=$(date '+%A, %d %B %Y')  # Organizing the date for a professional look
    CURRENT_TIME=$(date '+%I:%M:%S %p')   # Keeping track of the exact execution time

    # Extracting the distribution name from /etc/os-release

    DISTRO_NAME=$(grep -w "PRETTY_NAME" /etc/os-release | cut -d'"' -f2)

    # --- Display the system identity report ---
    echo ""
    echo "╔═══════════════════════════════════════════════════════════╗"
    echo "╠           OPEN SOURCE AUDIT — SYSTEM IDENTITY            ╣"
    echo "╚═══════════════════════════════════════════════════════════╣"
    echo "   Student    : $NAME"
    echo "   Software   : $SOFTWARE_CHOICE"
    echo "═══════════════════════════════════════════════════════════╣"
    echo "   Distribution : $DISTRO_NAME"
    echo "   Kernel       : $KERNEL"
    echo "   User         : $USER_NAME"
    echo "   Home Dir     : $HOME_DIR"
    echo "   Uptime       : $UPTIME"
    echo "   Date         : $CURRENT_DATE"
    echo "   Time         : $CURRENT_TIME"
    echo "═══════════════════════════════════════════════════════════╣"
    echo "   OS License   : GNU General Public License v2 (GPLv2)   ╣"
    echo "   The Linux kernel is licensed under GPLv2, which        ╣"
    echo "   guarantees the freedom to use, study, modify,          ╣"
    echo "═══════════════════════════════════════════════════════════╣"
    echo ""

# --- End of Script 1 ---
prade@PAVIPRADEEP:~$ bash scripts.sh
```

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Starter structure

```
prade@PAVIPRADEEP:~$
#!/bin/bash

# ------------------------------------------
# FOSS Package Inspector - Python
# This script checks if Python is installed,
# displays its version, and prints a description
# using a case statement.
# ------------------------------------------

# Step 1: Set default package name
pkg="python3"

# Step 2: Check if python3 command exists
echo ""
echo "╔═══════════════════════════════════════╗"
echo "╠═══         FOSS PACKAGE INSPECTOR    ═══╣"
echo "╠═══════════════════════════════════════╣"
echo "╚═  Inspecting package: $PACKAGE"
echo "╚═══════════════════════════════════════╝"
echo ""
# 'command -v' checks if the command is available in the system
if command -v "$pkg" >/dev/null 2>&1; then

    # If found, print success message
    echo "✅ $pkg is installed."

    # Step 3: Get Python version
    # '--version' prints version info
# --- End of Script 2 ---s NOT installed on this system."ion available."rogramming language used for web development, da

╔═══════════════════════════════════╗
╠═══         FOSS PACKAGE INSPECTOR    ═══╣
╠═══════════════════════════════════╣
╚═  Inspecting package:

✅ python3 is installed.
Version: Python 3.12.3
Description: Python is a high-level, interpreted programming language used for web development, data analysis, artificial intelligence, automation, and more
.
prade@PAVIPRADEEP:~$
```

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

```
prade@PAVIPRADEEP:~$ #!/bin/bash
# ==============================================================
# Script 3 (Modified): Disk and Permission Auditor (Python)
# Author: Rahul G | Course: Open Source Software
# Purpose: Audit system directories + Python configuration/setup
# ==============================================================

# --- Define important system directories ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

# --- Python-related paths ---
PYTHON_BIN=$(which python3 2>/dev/null)
PYTHON_CONFIG_DIR="$HOME/.config/python"
PYTHON_PIP_CONFIG="$HOME/.config/pip/pip.conf"

echo ""
echo "╔═══════════════════════════════════════════════════════════╗"
echo "╔           DISK AND PERMISSION AUDITOR (PYTHON)            ╗"
echo "╔═══════════════════════════════════════════════════════════╗"
echo "╔  Auditing system directories + Python environment         ╗"
echo "╔═══════════════════════════════════════════════════════════╗"
echo ""

# --- Overall disk usage ---
echo "=== Overall Disk Usage ==="
df -h / | awk 'NR==1 || NR==2'
echo ""

# --- Directory Audit ---
echo "=== Directory Audit Report ===="
echo "----------------------------------------------------------------"
printf "%-15s %-15s %-10s %-10s %s\n" "DIRECTORY" "PERMISSIONS" "OWNER" "GROUP" "SIZE"
echo "----------------------------------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        [ -z "$SIZE" ] && SIZE="N/A"
```

```
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        [ -z "$SIZE" ] && SIZE="N/A"

        printf "%-15s %-15s %-10s %-10s %s\n" "$DIR" "$PERMS" "$OWNER" "$GROUP" "$SIZE"
echo ""== Audit Complete ==="RSION" 2>&1)global.index-url <url>)"---"

╔═══════════════════════════════════════════════════════════╗
╔           DISK AND PERMISSION AUDITOR (PYTHON)            ╗
╔═══════════════════════════════════════════════════════════╗
╔  Auditing system directories + Python environment         ╗

=== Overall Disk Usage ===
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd       1007G  1.4G  955G   1% /

=== Directory Audit Report ===
----------------------------------------------------------------
DIRECTORY       PERMISSIONS     OWNER      GROUP      SIZE
----------------------------------------------------------------
/etc            drwxr-xr-x      root       root       4.3M
/var/log        drwxrwxr-x      root       syslog     184M
/home           drwxr-xr-x      root       root       56K
/usr/bin        drwxr-xr-x      root       root       94M
/tmp            drwxrwxrwt      root       root       24K

=== Python Configuration Audit ===
----------------------------------------------------------------
Python binary : /usr/bin/python3
Permissions   : lrwxrwxrwx
Owner         : root

Config dir    : /home/prade/.config/python [NOT FOUND]

Pip config    : /home/prade/.config/pip/pip.conf [NOT FOUND]
  (Create with: pip config set global.index-url <url>)

Python version: Python 3.12.3

=== Audit Complete ===

prade@PAVIPRADEEP:~$
```

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Starter structure

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary.

Use a while-read loop and an if-then inside it. Concepts to use: while read loop, if-then, counter variables, command-line arguments ($

```
$ #!/bin/bash

# ==============================================================
# Python Log File Analyzer
# Purpose: Read a log file, count keyword occurrences,
#          and display summary with last 5 matches
# ==============================================================

# --- Get input from user ---
# $1 → log file path
# $2 → keyword (default = Traceback)
LOGFILE="$1"
KEYWORD="${2:-Traceback}"

# --- Initialize counters ---
MATCH_COUNT=0
LINE_COUNT=0

# --- Check if input file is provided ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    exit 1
fi

# --- Check if file exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found!"
    exit 1
fi

# --- Check if file is empty ---
```

1).

Starter structure

```
# --- Check if file is empty ---
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: File is empty!"
    exit 0
fi

echo ""
echo "======================================"
echo "     PYTHON LOG FILE ANALYZER"
echo "======================================"
echo "File    : $LOGFILE"
echo "Keyword : $KEYWORD"
echo ""

# --- Read file line by line ---
while IFS= read -r LINE
do
    # Count total lines
    LINE_COUNT=$((LINE_COUNT + 1))

    # Check if line contains keyword
    if echo "$LINE" | grep -iq "$KEYWORD"
    then
        MATCH_COUNT=$((MATCH_COUNT + 1))
    fi

done < "$LOGFILE"

# --- Display summary ---
echo "========== Summary ==========="
echo "Total lines     : $LINE_COUNT"
```

```
# --- Show last 5 matching lines ---
echo "=== Last 5 Matches ===="
grep -in "$KEYWORD" "$LOGFILE" | tail -5

echo ""
echo "======= Completed ======="
$ #!/bin/bash
$ $ # ================================================================
$ $ # Python Log File Analyzer
$ $ # Purpose: Read a log file, count keyword occurrences,
$ $ #          and display summary with last 5 matches
$ $ # ================================================================
$ $ # --- Get input from user ---
$ $ # $1 → log file path
$ $ # $2 → keyword (default = Traceback)
$ $ LOGFILE="$1"
$ $ KEYWORD="${2:-Traceback}"
$ $ # --- Initialize counters ---
$ $ MATCH_COUNT=0
$ $ LINE_COUNT=0
$ $ # --- Check if input file is provided ---
$ $ if [ -z "$LOGFILE" ]; then
$ >
$ >     echo "Usage: $0 <logfile> [keyword]"
$ >
$ >     exit 1
$ >
$ > fi
$ Usage: bash <logfile> [keyword]
$ exit
$
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Starter structure

```
$ #!/bin/bash
# ==============================================================
# Script 5: Open Source Manifesto Generator
# ==============================================================

# --- Alias demonstration (Python-related) ---
# In bash, aliases are shortcuts for longer commands. For example:
# alias py='python3'              → typing 'py' runs 'python3'
# alias pipup='pip install --upgrade pip'
# alias runserver='python3 app.py'
# alias venv='python3 -m venv myenv'
# Aliases are typically defined in ~/.bashrc or ~/.bash_aliases
# They are best suited for interactive shells, not scripts.

# --- Gather user input ---
read -p "1. Name one open-source Python tool/library you use: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one Python project you would build and share: " BUILD

# --- Get date and user info ---
DATE=$(date '+%d %B %Y')
TIMESTAMP=$(date '+%Y%m%d_%H%M%S')
USERNAME=$(whoami)

# --- Output file ---
OUTPUT="manifesto_${USERNAME}_${TIMESTAMP}.txt"

# --- Write manifesto ---
echo "==================================================" > "$OUTPUT"
echo "   MY OPEN SOURCE MANIFESTO (PYTHON EDITION)" >> "$OUTPUT"
echo "   Generated on $DATE by $USERNAME" >> "$OUTPUT"
```

```
echo "Python and open source together empower developers to solve real-world problems efficien
echo "" >> "$OUTPUT"

echo "This is my manifesto. Written on $DATE. Signed by $USERNAME." >> "$OUTPUT"
echo "==================================================" >> "$OUTPUT"

# --- Display output ---
echo ""
echo "Your manifesto has been generated and saved."
echo ""
echo "--- BEGIN MANIFESTO ---"
cat "$OUTPUT"
echo "--- END MANIFESTO ---"
echo ""
echo "Saved to: $(pwd)/$OUTPUT"
echo ""
$ #!/bin/bash
$ $ # ==============================================================
$ $ # Script 5: Open Source Manifesto Generator
$ $ # ==============================================================
$ $ # --- Alias demonstration (Python-related) ---
$ $ # In bash, aliases are shortcuts for longer commands. For example:
$ $ # alias py='python3'              → typing 'py' runs 'python3'
$ $ # alias pipup='pip install --upgrade pip'
$ $ # alias runserver='python3 app.py'
$ $ # alias venv='python3 -m venv myenv'
$ $ # Aliases are typically defined in ~/.bashrc or ~/.bash_aliases
$ $ # They are best suited for interactive shells, not scripts.
$ $ # --- Gather user input ---
$ $ read -p "1. Name one open-source Python tool/library you use: " TOOL
$
```

Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- ☐ GNU Project — *The Free Software Definition*. Available at: https://www.gnu.org/philosophy/free-sw.html
- ☐ Open Source Initiative — *The Open Source Definition*. Available at: https://opensource.org/osd
- ☐ Eric S. Raymond — *The Cathedral and the Bazaar*. Available at: http://catb.org/~esr/writings/cathedral-bazaar/
- ☐ Linus Torvalds — *Just for Fun: The Story of an Accidental Revolutionary*

On Linux and shell scripting

☐ William Shotts — *The Linux Command Line*. Available at: http://linuxcommand.org

☐ GNU Project — *GNU Bash Manual*. Available at: https://www.gnu.org/software/bash/manual

☐ Linux Manual Pages — Available via man command in terminal or online at: http://man.cx

For finding license texts

☐ SPDX — *SPDX License List*. Available at: https://spdx.org/licenses

☐ Choose a License — Available at: https://choosealicense.com

☐ GitHub / GitLab — Project repositories (LICENSE files in respective repositories)